

TRACEFORGE V2 — PHASE 4

Phase: 4 of 4

Goal: Flask web dashboard + Docker deployment

Depends on: Phase 3 fully complete — all 36 tests passing, full pipeline working

Gate: Dashboard must display real data from the SQLite database before Docker is built

What Phase 4 builds

Two things:

1. **Flask dashboard** — a web interface that reads directly from the SQLite database built in Phase 3. Five pages: case list, case overview, IOC timeline, correlation view, report export.
2. **Docker deployment** — a Dockerfile and docker-compose.yml that packages the entire tool for portable deployment on any machine.

The dashboard does not replace the CLI. It reads the same database the CLI writes to. You run analysis via CLI, view and present findings via dashboard.

Step 4.1 — Install dashboard dependencies

```
cd ~/traceforge
source venv/bin/activate
pip install flask flask-cors
pip freeze > requirements.txt
```

Step 4.2 — Create the dashboard directory structure

```
mkdir -p ~/traceforge/dashboard/templates
mkdir -p ~/traceforge/dashboard/static/{css,js}
touch ~/traceforge/dashboard/__init__.py
```

Step 4.3 — Build the Flask app

File: `dashboard/app.py`

```
nano ~/traceforge/dashboard/app.py
```

```
"""
TraceForge v2 – Flask Dashboard
Reads directly from the SQLite database.
Run with: python -m dashboard.app
"""

from flask import Flask, render_template, jsonify, send_file, abort
from pathlib import Path
import json
import os
import sys

# Add project root to path
sys.path.insert(0, str(Path(__file__).parent.parent))

from traceforge.core.ledger import init_db, list_cases, get_case, get_evidence_log
from traceforge.core.store import get_artifacts, get_artifact_count
from traceforge.core.correlator import correlate, correlate_summary
from traceforge.core.timeline import build_timeline, timeline_to_dict
from traceforge.core.report import generate_report

app = Flask(__name__, template_folder="templates", static_folder="static")

REPORTS_DIR = Path.home() / "traceforge" / "reports"
```

```

@app.before_request
def setup():
    init_db()

# — ROUTES —

@app.route("/")
def index():
    """Case list – home page."""
    cases = list_cases()
    stats = []
    for c in cases:
        counts = get_artifact_count(c["id"])
        total = sum(counts.values())
        stats.append(**c, "artifact_count": total, "module
_counts": counts)
    return render_template("index.html", cases=stats)

@app.route("/case/<case_id>")
def case_overview(case_id):
    """Case overview – summary of all findings."""
    case = get_case(case_id)
    if not case:
        abort(404)
    evidence = get_evidence_log(case_id)
    counts = get_artifact_count(case_id)
    corr_results = correlate(case_id)
    corr_summary = correlate_summary(corr_results)
    timeline = build_timeline(case_id, corr_results)

    severity_counts = {}
    for e in timeline:
        severity_counts[e.severity] = severity_counts.get
(e.severity, 0) + 1

```

```

    return render_template("case.html",
        case=case,
        evidence=evidence,
        counts=counts,
        total_artifacts=sum(counts.values()),
        corr_summary=corr_summary,
        severity_counts=severity_counts,
        timeline_count=len(timeline)
    )

@app.route("/case/<case_id>/timeline")
def case_timeline(case_id):
    """Full IOC timeline view."""
    case = get_case(case_id)
    if not case:
        abort(404)
    corr_results = correlate(case_id)
    timeline = build_timeline(case_id, corr_results)
    return render_template("timeline.html",
        case=case,
        events=timeline_to_dict(timeline),
        total=len(timeline)
    )

@app.route("/case/<case_id>/correlations")
def case_correlations(case_id):
    """Correlation findings view."""
    case = get_case(case_id)
    if not case:
        abort(404)
    results = correlate(case_id)
    summary = correlate_summary(results)
    corr_list = [
        {
            "link_type": r.link_type,
            "confidence": r.confidence,

```

```

        "confidence_pct": int(r.confidence * 100),
        "description": r.description,
        "module_a": r.artifact_a.source_module,
        "type_a": r.artifact_a.artifact_type,
        "module_b": r.artifact_b.source_module,
        "type_b": r.artifact_b.artifact_type,
    }
    for r in results
]
return render_template("correlations.html",
    case=case,
    correlations=corr_list,
    summary=summary
)

@app.route("/case/<case_id>/report/<fmt>")
def export_report(case_id, fmt):
    """Generate and download a case report."""
    if fmt not in ("json", "html", "pdf"):
        abort(400)
    case = get_case(case_id)
    if not case:
        abort(404)
    try:
        paths = generate_report(case_id, formats=[fmt])
        if fmt in paths:
            return send_file(paths[fmt], as_attachment=True)
    except Exception as e:
        abort(500)
    abort(500)

# — API ENDPOINTS —

```

```

@app.route("/api/cases")

```

```

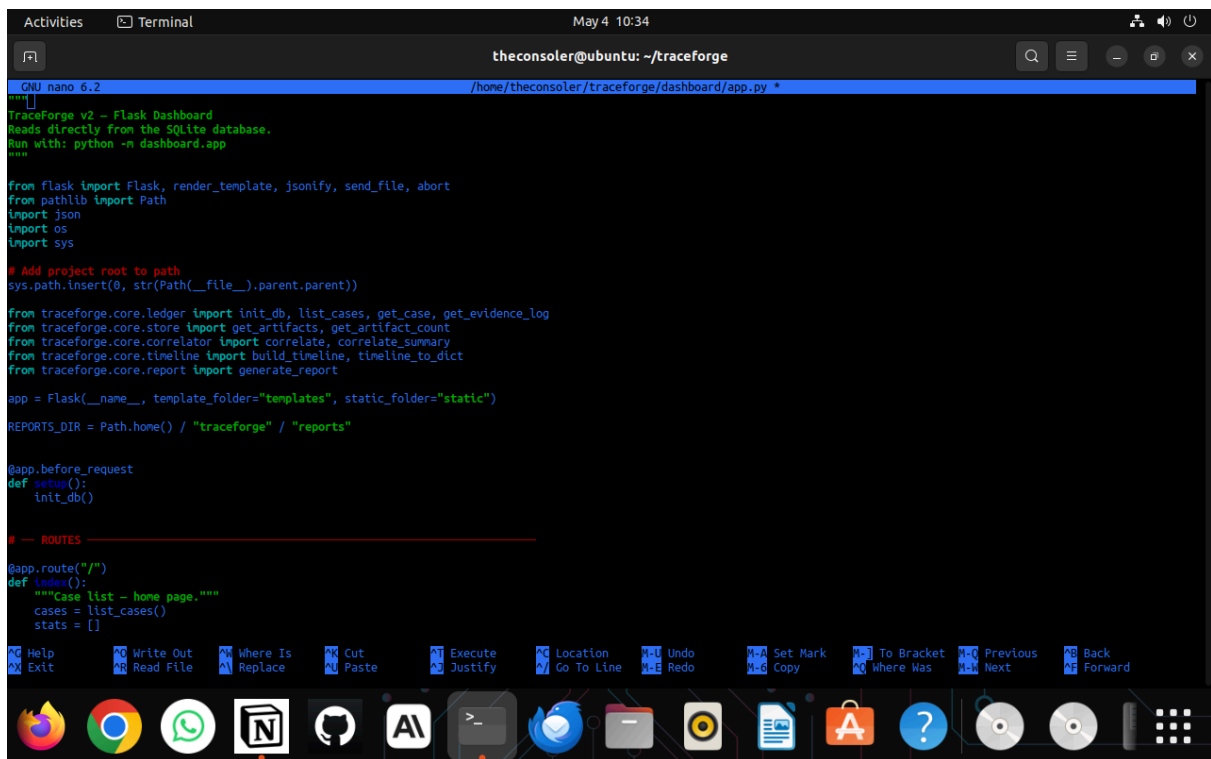
def api_cases():
    cases = list_cases()
    return jsonify(cases)

@app.route("/api/case/<case_id>/timeline")
def api_timeline(case_id):
    corr_results = correlate(case_id)
    timeline = build_timeline(case_id, corr_results)
    return jsonify(timeline_to_dict(timeline))

@app.route("/api/case/<case_id>/artifacts")
def api_artifacts(case_id):
    artifacts = get_artifacts(case_id)
    return jsonify([a.to_dict() for a in artifacts])

if __name__ == "__main__":
    print("\n TraceForge v2 Dashboard")
    print(" Running at: http://0.0.0.0:5000")
    print(" Press Ctrl+C to stop\n")
    app.run(debug=True, host="0.0.0.0", port=5000)

```



```
GNU nano 6.2 /home/theconsoler/traceforge/dashboard/app.py
TraceForge v2 - Flask Dashboard
Reads directly from the SQLite database.
Run with: python -m dashboard.app

from flask import Flask, render_template, jsonify, send_file, abort
from pathlib import Path
import json
import os
import sys

# Add project root to path
sys.path.insert(0, str(Path(__file__).parent.parent))

from traceforge.core.ledger import init_db, list_cases, get_case, get_evidence_log
from traceforge.core.store import get_artifacts, get_artifact_count
from traceforge.core.correlator import correlate, correlate_summary
from traceforge.core.timeline import build_timeline, timeline_to_dict
from traceforge.core.report import generate_report

app = Flask(__name__, template_folder="templates", static_folder="static")
REPORTS_DIR = Path.home() / "traceforge" / "reports"

@app.before_request
def setup():
    init_db()

# --- ROUTES ---
@app.route("/")
def index():
    """Case list - home page."""
    cases = list_cases()
    stats = []
```

Step 4.4 — Build the base template

File: `dashboard/templates/base.html`

```
nano ~/traceforge/dashboard/templates/base.html
```

```
<!DOCTYPE html>
<html lang="en">
<head>
<meta charset="UTF-8">
<meta name="viewport" content="width=device-width, initial-
scale=1.0">
<title>{% block title %}TraceForge v2{% endblock %}</title>
<style>
    :root {
        --bg:          #0d1117;
        --surface:     #161b22;
        --border:       #21262d;
        --text:         #e6edf3;
        --muted:        #8b949e;
        --cyan:         #22d3ee;
```

```

    --purple:    #a78bfa;
    --green:     #4ade80;
    --red:       #f87171;
    --yellow:    #fbbf24;
    --orange:    #fb923c;
  }
  * { margin:0; padding:0; box-sizing:border-box; }
  body { background:var(--bg); color:var(--text); font-family:'Segoe UI',Arial,sans-serif; min-height:100vh; }

  /* NAV */
  nav { background:var(--surface); border-bottom:1px solid var(--border); padding:0 24px; display:flex; align-items:center; gap:24px; height:56px; }
  nav .brand { color:var(--cyan); font-weight:700; font-size:16px; text-decoration:none; letter-spacing:1px; }
  nav a { color:var(--muted); text-decoration:none; font-size:13px; }
  nav a:hover { color:var(--text); }

  /* LAYOUT */
  .container { max-width:1200px; margin:0 auto; padding:32px 24px; }
  .page-title { font-size:22px; font-weight:600; margin-bottom:24px; }
  .page-title span { color:var(--cyan); }

  /* CARDS */
  .card { background:var(--surface); border:1px solid var(--border); border-radius:10px; padding:20px; margin-bottom:16px; }
  .card-title { font-size:13px; font-weight:600; color:var(--muted); text-transform:uppercase; letter-spacing:0.5px; margin-bottom:16px; }

  /* STATS GRID */
  .stats { display:grid; grid-template-columns:repeat(auto-fit, minmax(160px,1fr)); gap:16px; margin-bottom:24px; }

```



```

    .stat { background:var(--surface); border:1px solid var(--border); border-radius:10px; padding:20px; text-align:center; }
    .stat-number { font-size:32px; font-weight:700; color:var(--cyan); line-height:1; }
    .stat-label { font-size:12px; color:var(--muted); margin-top:6px; }

    /* TABLE */
    table { width:100%; border-collapse:collapse; font-size:13px; }
    th { color:var(--muted); text-align:left; padding:10px 12px; border-bottom:1px solid var(--border); font-weight:500; font-size:12px; text-transform:uppercase; }
    td { padding:10px 12px; border-bottom:1px solid var(--border); vertical-align:top; }
    tr:last-child td { border-bottom:none; }
    tr:hover td { background:rgba(255,255,255,0.02); }

    /* BADGES */
    .badge { display:inline-block; padding:2px 8px; border-radius:20px; font-size:11px; font-weight:500; }
    .badge-memory { background:#2d1b69; color:#a78bfa; }
    .badge-disk { background:#0c3251; color:#7dd3fc; }
    .badge-logs { background:#052e16; color:#4ade80; }
    .badge-network { background:#450a0a; color:#f87171; }
    .badge-critical { background:#450a0a; color:#f87171; }
    .badge-high { background:#431407; color:#fb923c; }
    .badge-medium { background:#422006; color:#fbbf24; }
    .badge-low { background:#0c1a3b; color:#7dd3fc; }
    .badge-info { background:#1c2333; color:#8b949e; }

    /* BUTTONS */
    .btn { display:inline-block; padding:8px 16px; border-radius:6px; font-size:13px; font-weight:500; text-decoration:none; cursor:pointer; border:none; }
    .btn-primary { background:var(--cyan); color:#0d1117; }
    .btn-primary:hover { opacity:0.9; }

```

```

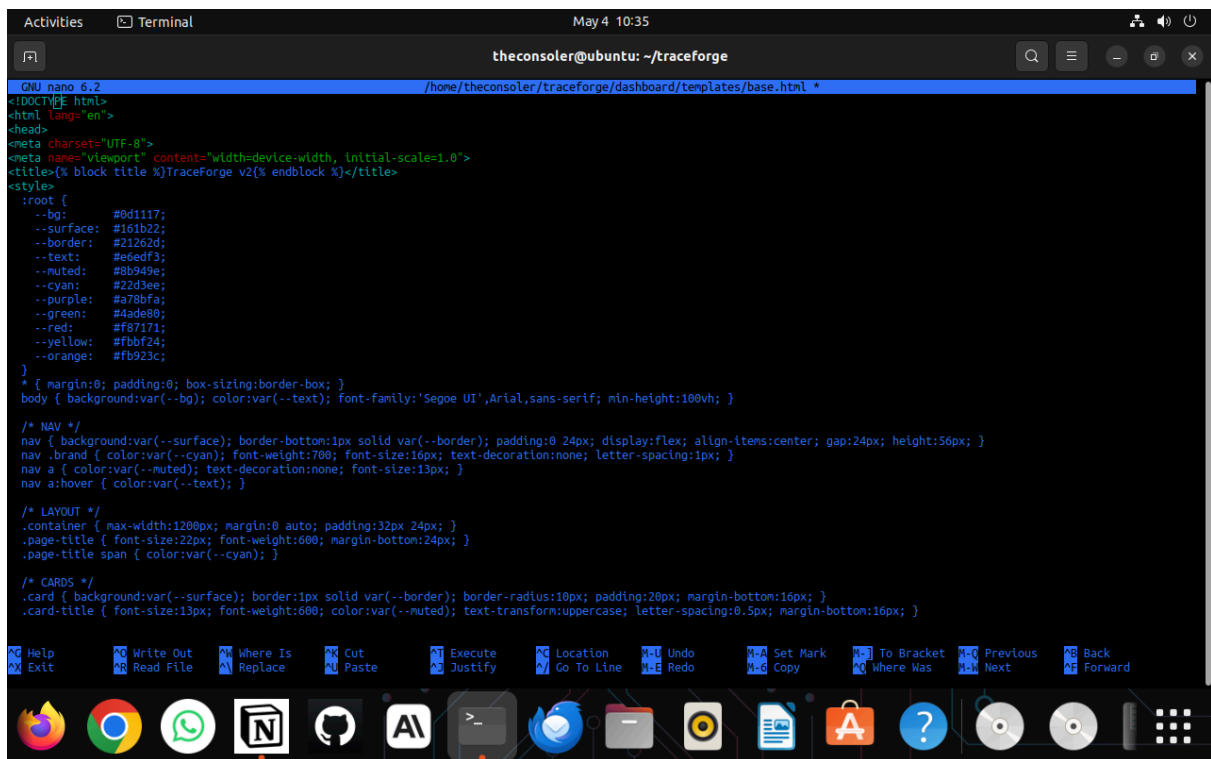
    .btn-ghost { background:transparent; border:1px solid var
(--border); color:var(--text); }
    .btn-ghost:hover { border-color:var(--cyan); color:var(--
cyan); }

    /* BACK LINK */
    .back { display:inline-flex; align-items:center; gap:6px;
color:var(--muted); text-decoration:none; font-size:13px; m
argin-bottom:20px; }
    .back:hover { color:var(--cyan); }

    /* HASH */
    .hash { font-family:monospace; font-size:11px; color:var
(--muted); word-break:break-all; }

    /* EMPTY STATE */
    .empty { text-align:center; padding:60px 20px; color:var
(--muted); }
    .empty h3 { font-size:16px; margin-bottom:8px; color:var
(--text); }
</style>
</head>
<body>
<nav>
    <a href="/" class="brand">< TRACEFORGE v2</a>
    <a href="/">Cases</a>
    <span style="flex:1"></span>
    <span style="font-size:12px;color:var(--muted);">Digital
Forensics & Incident Response</span>
</nav>
{% block content %}{% endblock %}
</body>
</html>

```



Step 4.5 — Build the case list page

File: `dashboard/templates/index.html`

```
nano ~/traceforge/dashboard/templates/index.html
```

```

{% extends "base.html" %}
{% block title %}TraceForge v2 – Cases{% endblock %}
{% block content %}
<div class="container">
  <div style="display:flex;align-items:center;justify-content:space-between;margin-bottom:24px;">
    <h1 class="page-title" style="margin:0;">Investigation
  <span>Cases</span></h1>
  </div>

  {% if cases %}
  <div class="card" style="padding:0;">
    <table>
      <thead>
        <tr>

```

```

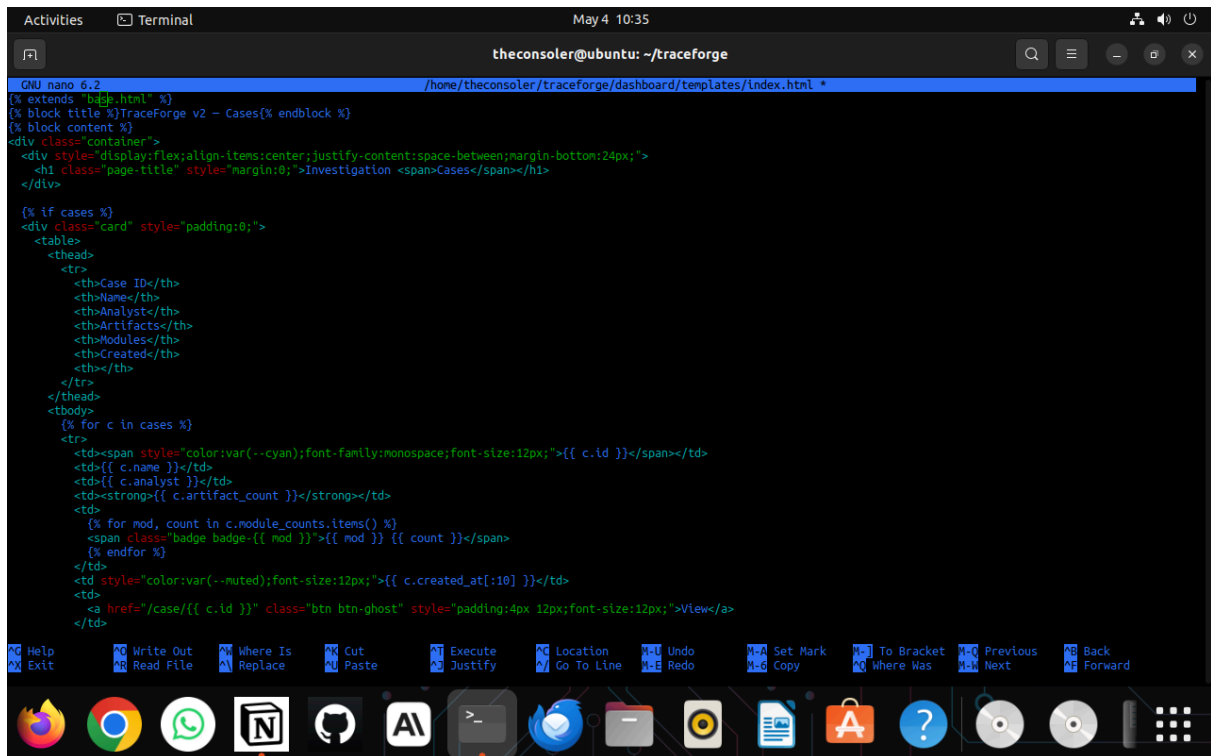
        <th>Case ID</th>
        <th>Name</th>
        <th>Analyst</th>
        <th>Artifacts</th>
        <th>Modules</th>
        <th>Created</th>
        <th></th>
    </tr>
</thead>
<tbody>
    {% for c in cases %}
    <tr>
        <td><span style="color:var(--cyan);font-family:monospace;font-size:12px;">{{ c.id }}</span></td>
        <td>{{ c.name }}</td>
        <td>{{ c.analyst }}</td>
        <td><strong>{{ c.artifact_count }}</strong></td>
        <td>
            {% for mod, count in c.module_counts.items() %}
            <span class="badge badge-{{ mod }}">{{ mod }}
            {{ count }}</span>
            {% endfor %}
        </td>
        <td style="color:var(--muted);font-size:12px;">{{
c.created_at[:10] }}</td>
        <td>
            <a href="/case/{{ c.id }}" class="btn btn-ghost" style="padding:4px 12px;font-size:12px;">View</a>
        </td>
    </tr>
    {% endfor %}
</tbody>
</table>
</div>
{% else %}
<div class="empty">
    <h3>No cases yet</h3>
    <p>Create a case using the CLI: <code>python -m tracefo

```

```

rge.cli case new</code></p>
</div>
{% endif %}
</div>
{% endblock %}

```



Step 4.6 — Build the case overview page

File: `dashboard/templates/case.html`

```
nano ~/traceforge/dashboard/templates/case.html
```

```

{% extends "base.html" %}
{% block title %}{{ case.id }} – TraceForge v2{% endblock %}
{% block content %}
<div class="container">
  <a href="/" class="back">← All Cases</a>

  <div style="display:flex;align-items:flex-start;justify-content:space-between;margin-bottom:24px;">

```

```

<div>
  <h1 class="page-title" style="margin-bottom:4px;">Case: <span>{{ case.id }}</span></h1>
  <p style="color:var(--muted);font-size:13px;">{{ case.name }} &nbsp;&nbsp;&nbsp; Analyst: {{ case.analyst }} &nbsp;&nbsp;&nbsp; {{ case.created_at[:19] }}</p>
</div>
<div style="display:flex;gap:8px;">
  <a href="/case/{{ case.id }}/timeline" class="btn btn-ghost">Timeline</a>
  <a href="/case/{{ case.id }}/correlations" class="btn btn-ghost">Correlations</a>
  <a href="/case/{{ case.id }}/report/html" class="btn btn-primary">Export HTML</a>
  <a href="/case/{{ case.id }}/report/pdf" class="btn btn-primary">Export PDF</a>
  <a href="/case/{{ case.id }}/report/json" class="btn btn-ghost">Export JSON</a>
</div>
</div>

<!-- Stats -->
<div class="stats">
  <div class="stat">
    <div class="stat-number">{{ total_artifacts }}</div>
    <div class="stat-label">Total Artifacts</div>
  </div>
  <div class="stat">
    <div class="stat-number">{{ corr_summary.total }}</div>
    <div class="stat-label">Correlations</div>
  </div>
  <div class="stat">
    <div class="stat-number" style="color:var(--red);">{{ severity_counts.get('critical', 0) }}</div>
    <div class="stat-label">Critical</div>
  </div>
  <div class="stat">

```

```

        <div class="stat-number" style="color:var(--orange);">{{ severity_counts.get('high', 0) }}</div>
        <div class="stat-label">High</div>
    </div>
    <div class="stat">
        <div class="stat-number" style="color:var(--yellow);">{{ severity_counts.get('medium', 0) }}</div>
        <div class="stat-label">Medium</div>
    </div>
    <div class="stat">
        <div class="stat-number">{{ timeline_count }}</div>
        <div class="stat-label">Timeline Events</div>
    </div>
</div>

<!-- Artifacts by module -->
<div class="card">
    <div class="card-title">Artifacts by Module</div>
    <table>
        <thead><tr><th>Module</th><th>Artifact Count</th><th>
</th></tr></thead>
        <tbody>
            {% for mod, count in counts.items() %}
            <tr>
                <td><span class="badge badge-{{ mod }}">{{ mod.upper() }}</span></td>
                <td>{{ count }}</td>
                <td style="color:var(--muted);font-size:12px;">
                    {% if mod == 'memory' %}processes, connections,
commands
                    {% elif mod == 'disk' %}files, deleted files, m
etadata
                    {% elif mod == 'logs' %}logins, sudo, brute for
ce
                    {% elif mod == 'network' %}DNS, HTTP, TCP flows
                    {% endif %}
                </td>
            </tr>
        </tbody>
    </table>
</div>

```

```

        {% endfor %}
    </tbody>
</table>
</div>

<!-- Evidence ledger -->
<div class="card">
    <div class="card-title">Evidence Ledger – Chain of Cust
ody</div>
    {% if evidence %}
    <table>
        <thead><tr><th>File</th><th>Module</th><th>SHA-256</t
h><th>Size</th><th>Recorded</th></tr></thead>
        <tbody>
            {% for e in evidence %}
            <tr>
                <td style="font-family:monospace;font-size:12p
x;">{{ e.file_path.split('/')[ -1] }}</td>
                <td><span class="badge badge-{{ e.module }}">{{
e.module.upper() }}</span></td>
                <td class="hash">{{ e.sha256_hash[:32] }}...</td>
                <td style="color:var(--muted);font-size:12px;">{{
'{:,}'.format(e.file_size_bytes) }} B</td>
                <td style="color:var(--muted);font-size:12px;">{{
e.recorded_at[:19] }}</td>
            </tr>
            {% endfor %}
        </tbody>
    </table>
    {% else %}
    <p style="color:var(--muted);font-size:13px;">No eviden
ce logged yet.</p>
    {% endif %}
</div>

<!-- Correlation summary -->
{% if corr_summary.total > 0 %}
<div class="card">

```



```

<div class="card-title">Correlation Summary</div>
<div style="display:flex;gap:24px;">
  <div><span style="color:var(--red);font-size:20px;font-weight:700;">{{ corr_summary.high_confidence }}</span><br><span style="font-size:12px;color:var(--muted);">High confidence</span></div>
  <div><span style="color:var(--yellow);font-size:20px;font-weight:700;">{{ corr_summary.medium_confidence }}</span><br><span style="font-size:12px;color:var(--muted);">Medium confidence</span></div>
  <div><span style="color:var(--muted);font-size:20px;font-weight:700;">{{ corr_summary.low_confidence }}</span><br><span style="font-size:12px;color:var(--muted);">Low confidence</span></div>
</div>
<div style="margin-top:12px;">
  <a href="/case/{{ case.id }}/correlations" class="btn btn-ghost">View all correlations →</a>
</div>
</div>
{% endif %}
</div>
{% endblock %}

```

Step 4.7 — Build the timeline page

File: `dashboard/templates/timeline.html`

```
nano ~/traceforge/dashboard/templates/timeline.html
```

```

{% extends "base.html" %}
{% block title %}Timeline – {{ case.id }}{% endblock %}
{% block content %}
<div class="container">
  <a href="/case/{{ case.id }}" class="back">← Case Overview</a>
  <h1 class="page-title">IOC Timeline – <span>{{ case.id }}

```

```

</span></h1>
<p style="color:var(--muted);font-size:13px;margin-bottom:24px;">{{ total }} events sorted chronologically across all modules</p>

<!-- Filters -->
<div style="display:flex;gap:8px;margin-bottom:20px;flex-wrap:wrap;">
  <button class="btn btn-ghost filter-btn active" data-filter="all">All</button>
  <button class="btn btn-ghost filter-btn" data-filter="memory" style="color:#a78bfa;">Memory</button>
  <button class="btn btn-ghost filter-btn" data-filter="disk" style="color:#7dd3fc;">Disk</button>
  <button class="btn btn-ghost filter-btn" data-filter="logs" style="color:#4ade80;">Logs</button>
  <button class="btn btn-ghost filter-btn" data-filter="network" style="color:#f87171;">Network</button>
  <button class="btn btn-ghost filter-btn" data-filter="critical" style="color:#f87171;">Critical</button>
  <button class="btn btn-ghost filter-btn" data-filter="high" style="color:#fb923c;">High</button>
</div>

<!-- Timeline -->
<div class="card" style="padding:0;">
  <table id="timeline-table">
    <thead>
      <tr>
        <th style="width:150px;">Timestamp</th>
        <th style="width:100px;">Module</th>
        <th style="width:80px;">Severity</th>
        <th style="width:120px;">Type</th>
        <th>Summary</th>
      </tr>
    </thead>
    <tbody>
      {% for e in events %}

```

```

        <tr class="tl-row" data-module="{{ e.module }}" data-severity="{{ e.severity }}">
            <td style="font-family:monospace;font-size:11px;color:var(--muted);">{{ e.timestamp[:19] }}</td>
            <td><span class="badge badge-{{ e.module }}">{{ e.module.upper() }}</span></td>
            <td><span class="badge badge-{{ e.severity }}">{{ e.severity.upper() }}</span></td>
            <td style="font-size:12px;color:var(--muted);">{{ e.artifact_type }}</td>
            <td style="font-size:13px;">
                {{ e.summary }}
                {% if e.correlation_links %}
                <div style="font-size:11px;color:var(--cyan);margin-top:3px;">> {{ e.correlation_links[0] }}</div>
                {% endif %}
            </td>
        </tr>
    {% endfor %}
</tbody>
</table>
</div>
</div>

<script>
document.querySelectorAll('.filter-btn').forEach(btn => {
    btn.addEventListener('click', () => {
        document.querySelectorAll('.filter-btn').forEach(b =>
b.classList.remove('active'));
        btn.classList.add('active');
        const f = btn.dataset.filter;
        document.querySelectorAll('.tl-row').forEach(row => {
            if (f === 'all') {
                row.style.display = '';
            } else if (f === 'critical' || f === 'high' || f ===
'medium' || f === 'low') {
                row.style.display = row.dataset.severity === f ? ''
: 'none';
            }
        });
    });
});

```

```

    } else {
        row.style.display = row.dataset.module === f ? '':
'none';
    }
    });
});
});
</script>
{% endblock %}

```

```

GNU nano 6.2 /home/theconsole/traceforge/dashboard/templates/case.html
{% extends "base.html" %}
{% block title %}{{ case.id }} - TraceForge v2{% endblock %}
{% block content %}
<div class="container">
  <a href="/" class="back">← All Cases</a>

  <div style="display:flex;align-items:flex-start;justify-content:space-between;margin-bottom:24px;">
    <div>
      <h1 class="page-title" style="margin-bottom:4px;">Case: <span>{{ case.id }}</span></h1>
      <p style="color:var(--muted);font-size:13px;">{{ case.name }} &nbsp;&nbsp;&nbsp; Analyst: {{ case.analyst }} &nbsp;&nbsp;&nbsp; {{ case.created_at[:19] }}</p>
    </div>
    <div style="display:flex;gap:8px;">
      <a href="/case/{{ case.id }}/timeline" class="btn btn-ghost">Timeline</a>
      <a href="/case/{{ case.id }}/correlations" class="btn btn-ghost">Correlations</a>
      <a href="/case/{{ case.id }}/report/html" class="btn btn-primary">Export HTML</a>
      <a href="/case/{{ case.id }}/report/pdf" class="btn btn-primary">Export PDF</a>
      <a href="/case/{{ case.id }}/report/json" class="btn btn-ghost">Export JSON</a>
    </div>
  </div>

  <!-- Stats -->
  <div class="stats">
    <div class="stat">
      <div class="stat-number">{{ total_artifacts }}</div>
      <div class="stat-label">Total Artifacts</div>
    </div>
    <div class="stat">
      <div class="stat-number">{{ corr_summary.total }}</div>
      <div class="stat-label">Correlations</div>
    </div>
    <div class="stat">
      <div class="stat-number" style="color:var(--red);">{{ severity_counts.get('critical', 0) }}</div>
      <div class="stat-label">Critical</div>
    </div>
    <div class="stat">
      <div class="stat-number" style="color:var(--orange);">{{ severity_counts.get('high', 0) }}</div>
      <div class="stat-label">High</div>
    </div>
  </div>
</div>

```

Step 4.8 — Build the correlations page

File: `dashboard/templates/correlations.html`

```
nano ~/traceforge/dashboard/templates/correlations.html
```

```

{% extends "base.html" %}
{% block title %}Correlations - {{ case.id }}{% endblock %}
{% block content %}
<div class="container">
  <a href="/case/{{ case.id }}" class="back">← Case Overview

```

```

w</a>
<h1 class="page-title">Correlation Findings – <span>{{ ca
se.id }}</span></h1>

<div class="stats" style="margin-bottom:24px;">
  <div class="stat">
    <div class="stat-number">{{ summary.total }}</div>
    <div class="stat-label">Total Links</div>
  </div>
  <div class="stat">
    <div class="stat-number" style="color:var(--red);">{{
summary.high_confidence }}</div>
    <div class="stat-label">High Confidence</div>
  </div>
  <div class="stat">
    <div class="stat-number" style="color:var(--yellow);">{{ summary.medium_confidence }}</div>
    <div class="stat-label">Medium Confidence</div>
  </div>
</div>

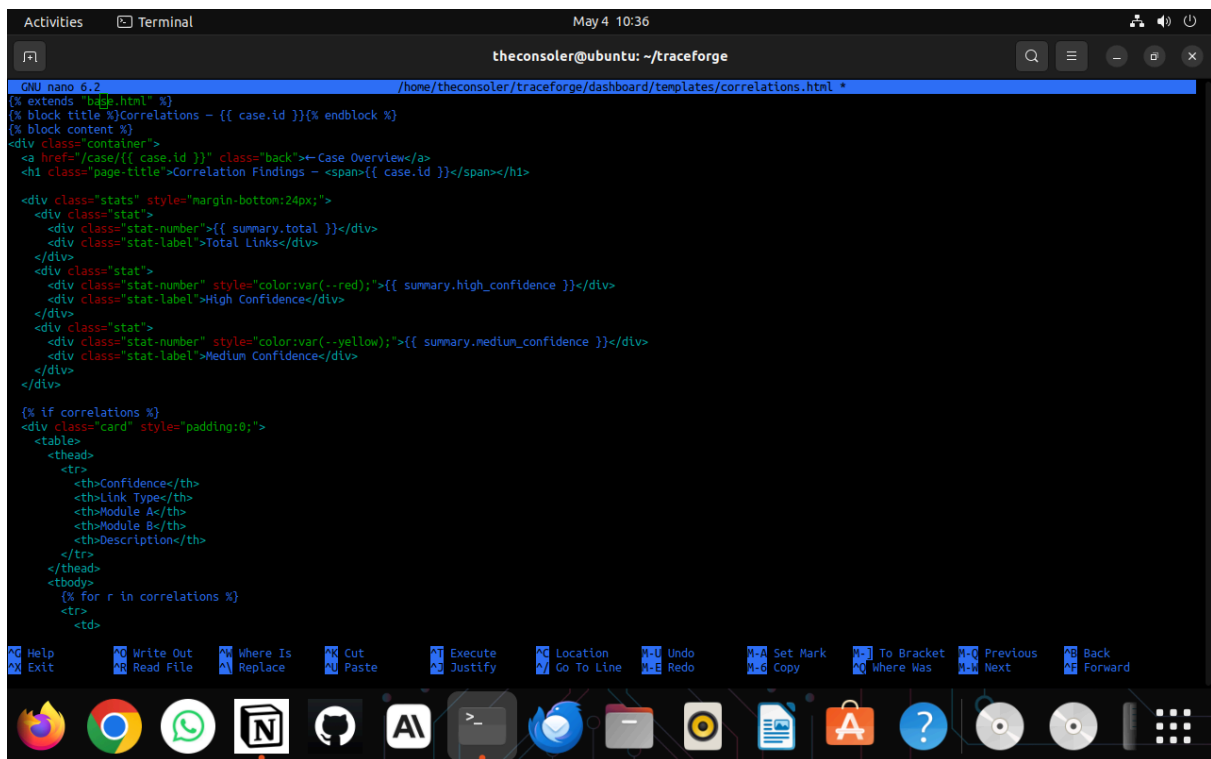
{% if correlations %}
<div class="card" style="padding:0;">
  <table>
    <thead>
      <tr>
        <th>Confidence</th>
        <th>Link Type</th>
        <th>Module A</th>
        <th>Module B</th>
        <th>Description</th>
      </tr>
    </thead>
    <tbody>
      {% for r in correlations %}
      <tr>
        <td>
          <span class="badge badge-{{ if r.confidence_pct

```

```

>= 80 %}critical{% elif r.confidence_pct >= 50 %}medium{% e
lse %}low{% endif %}">
    {{ r.confidence_pct }}%
    </span>
</td>
<td style="font-size:12px;color:var(--muted);">{{
r.link_type }}</td>
<td><span class="badge badge-{{ r.module_a }}">{{
r.module_a.upper() }}</span> <span style="font-size:11px;co
lor:var(--muted);">{{ r.type_a }}</span></td>
<td><span class="badge badge-{{ r.module_b }}">{{
r.module_b.upper() }}</span> <span style="font-size:11px;co
lor:var(--muted);">{{ r.type_b }}</span></td>
<td style="font-size:13px;">{{ r.description }}</
td>
</tr>
{% endfor %}
</tbody>
</table>
</div>
{% else %}
<div class="empty">
    <h3>No correlations found</h3>
    <p>Run analysis on multiple modules and then run the co
rrelate command.</p>
</div>
{% endif %}
</div>
{% endblock %}

```



```
GNU nano 6.2 /home/theconsoler/traceforge/dashboard/templates/correlations.html *
{% extends "base.html" %}
{% block title %}Correlations - {{ case.id }}{% endblock %}
{% block content %}
<div class="container">
  <a href="/case/{{ case.id }}" class="back">← Case Overview</a>
  <h1 class="page-title">Correlation Findings - <span>{{ case.id }}</span></h1>

  <div class="stats" style="margin-bottom:24px;">
    <div class="stat">
      <div class="stat-number">{{ summary.total }}</div>
      <div class="stat-label">Total Links</div>
    </div>
    <div class="stat">
      <div class="stat-number" style="color:var(--red);">{{ summary.high_confidence }}</div>
      <div class="stat-label">High Confidence</div>
    </div>
    <div class="stat">
      <div class="stat-number" style="color:var(--yellow);">{{ summary.medium_confidence }}</div>
      <div class="stat-label">Medium Confidence</div>
    </div>
  </div>

  {% if correlations %}
  <div class="card" style="padding:0;">
    <table>
      <thead>
        <tr>
          <th>Confidence</th>
          <th>Link Type</th>
          <th>Module A</th>
          <th>Module B</th>
          <th>Description</th>
        </tr>
      </thead>
      <tbody>
        {% for r in correlations %}
        <tr>
          <td>

```

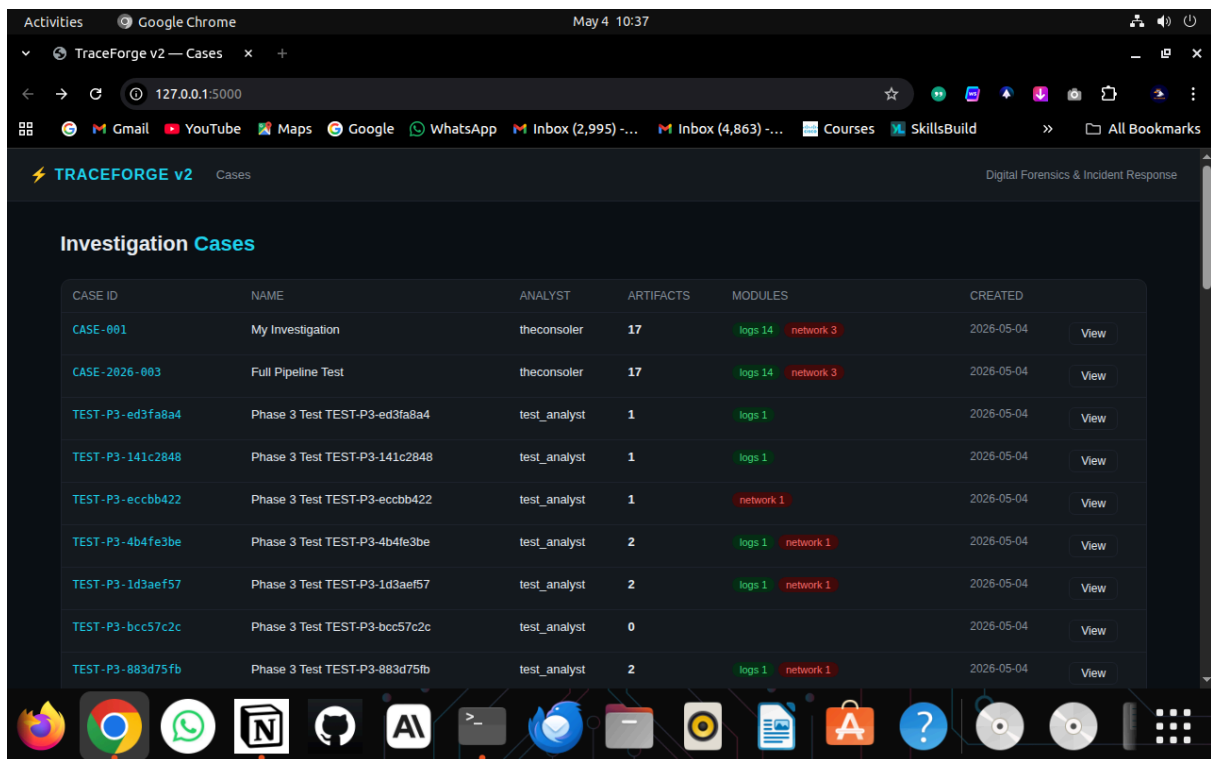
Step 4.9 — Run the dashboard

```
cd ~/traceforge
source venv/bin/activate
python -m dashboard.app
```

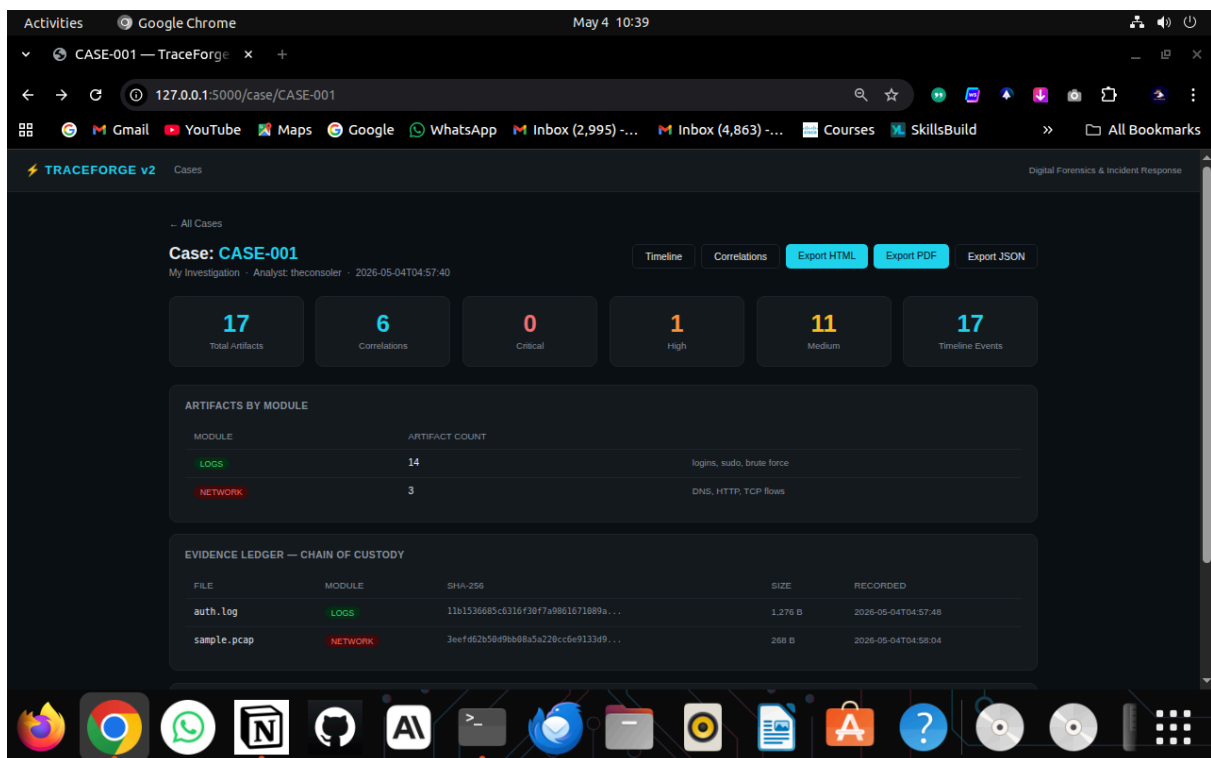
Open your browser and go to:

```
http://127.0.0.1:5000
```

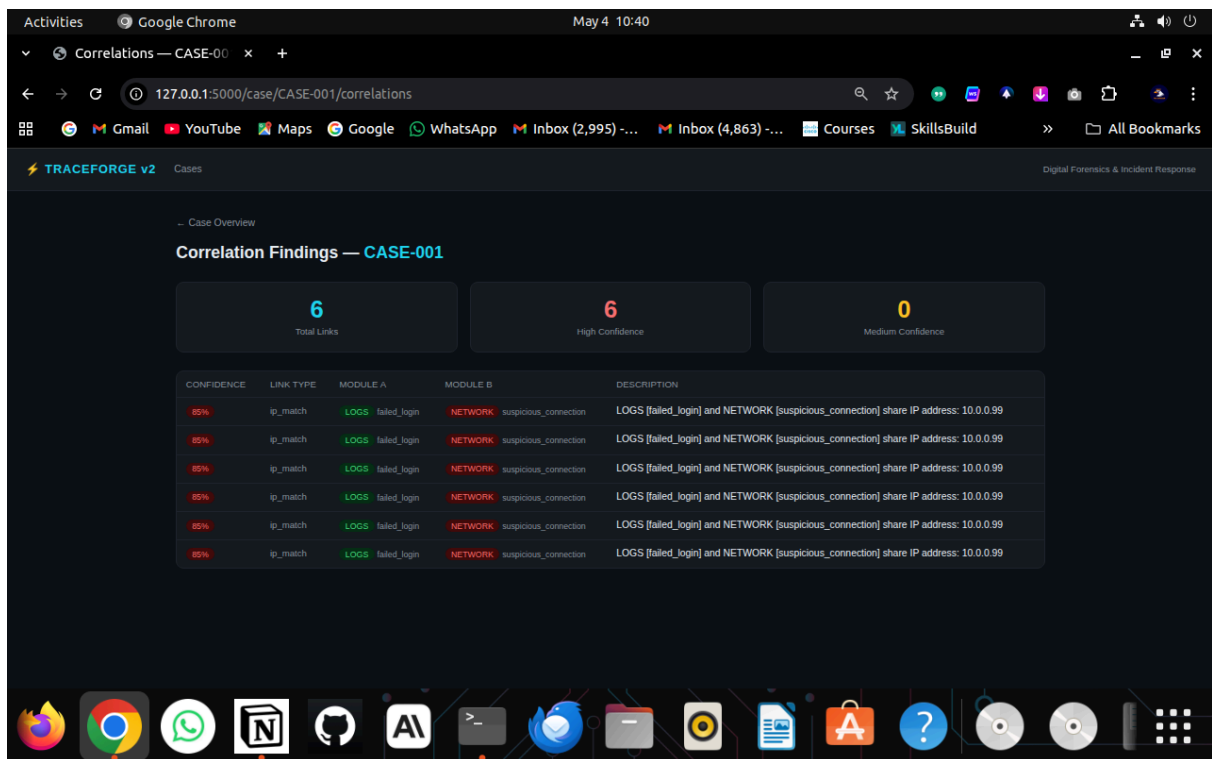
You should see your existing cases listed. Click into any case to see the full overview, timeline, and correlations.



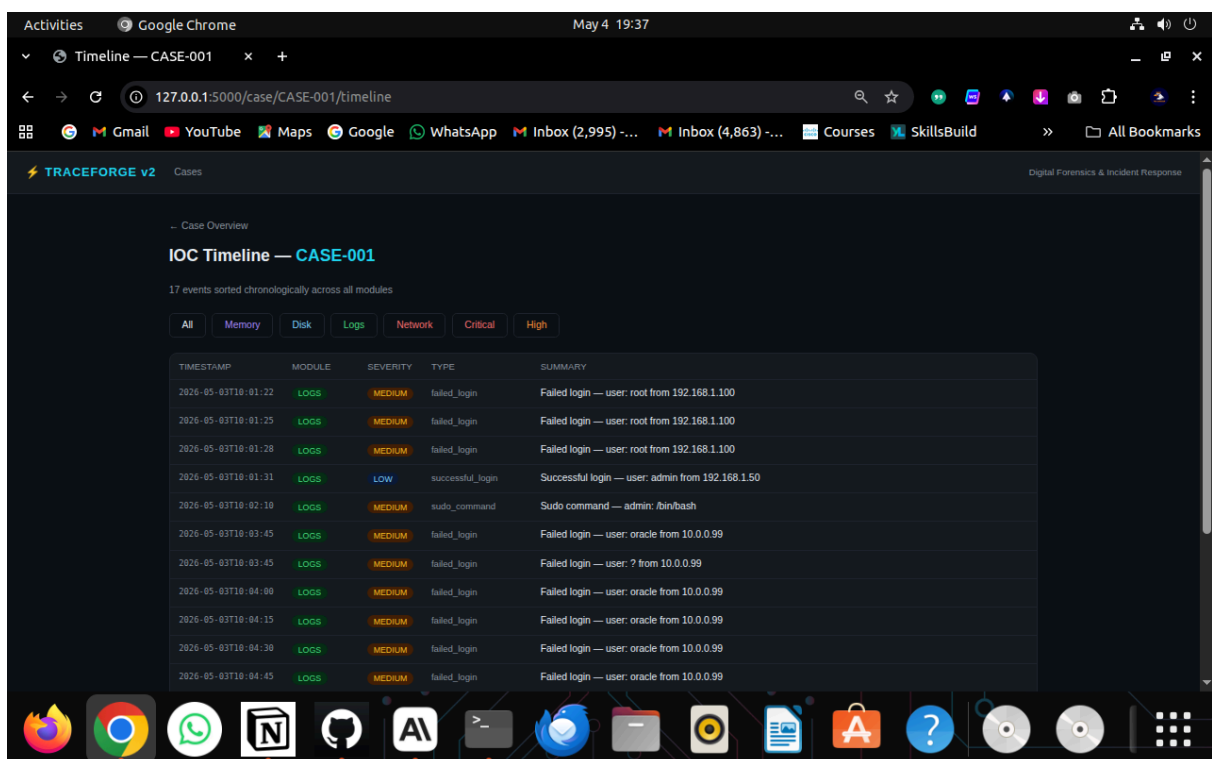
This is the dashboard which shows all the existing test cases or even some conducted sample cases.



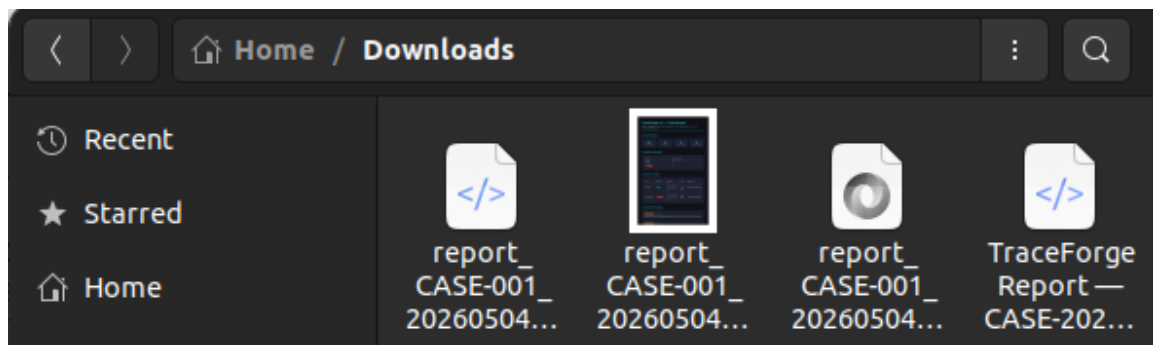
Also I have opened a case and you can see all the details that explains about the particular case.



Some correlation findings of case one.



Here are the timelines of the CASE-001



You can also look on to the downloaded files that I have made it from the dashboard of the CASE 001.

Step 4.10 — Build the Dockerfile

```
nano ~/traceforge/Dockerfile
```

```
FROM ubuntu:22.04

ENV DEBIAN_FRONTEND=noninteractive
ENV PYTHONDONTWRITEBYTECODE=1
ENV PYTHONUNBUFFERED=1

# System dependencies
RUN apt-get update && apt-get install -y \
    build-essential \
    python3.11 \
    python3.11-venv \
    python3.11-dev \
    python3-pip \
    git \
    tshark \
    sleuthkit \
    libewf-dev \
    libvhdi-dev \
    libvmrk-dev \
    libbde-dev \
    libsmdev-dev \
    libffi-dev \
```

```
libssl-dev \  
pkg-config \  
libpango-1.0-0 \  
libpangoft2-1.0-0 \  
wget \  
curl \  
&& rm -rf /var/lib/apt/lists/  
  
# Create working directory  
WORKDIR /traceforge  
  
# Copy requirements and install  
COPY requirements.txt .  
RUN python3.11 -m pip install --upgrade pip && \  
    python3.11 -m pip install -r requirements.txt  
  
# Copy project  
COPY . .  
  
# Create data and reports directories  
RUN mkdir -p /traceforge/data /traceforge/reports /tracefor  
ge/samples  
  
# Default command – show help  
ENTRYPOINT ["python3.11", "-m", "traceforge.cli"]  
CMD ["--help"]
```



```
Activities Terminal May 4 10:43
theconsoler@ubuntu: ~/traceforge
GNU nano 6.2 /home/theconsoler/traceforge/Dockerfile *
FROM ubuntu:22.04

ENV DEBIAN_FRONTEND=noninteractive
ENV PYTHONDONTWRITEBYTECODE=1
ENV PYTHONUNBUFFERED=1

# System dependencies
RUN apt-get update && apt-get install -y \
    build-essential \
    python3.11 \
    python3.11-venv \
    python3.11-dev \
    python3-pip \
    git \
    tshark \
    sleuthkit \
    libewf-dev \
    libvhdi-dev \
    libvmtoolsd-dev \
    libbde-dev \
    libsnappy-dev \
    libffi-dev \
    libssl-dev \
    pkg-config \
    libpango-1.0-0 \
    libpangot2-1.0-0 \
    wget \
    curl \
    && rm -rf /var/lib/apt/lists/*

# Create working directory
WORKDIR /traceforge

# Copy requirements and install
COPY requirements.txt .
RUN python3.11 -m pip install --upgrade pip && \
    python3.11 -m pip install -r requirements.txt
```

```
cat > ~/traceforge/docker-compose.yml << 'EOF'
services:
  traceforge:
    build: .
    container_name: traceforge-cli
    volumes:
      - ./data:/traceforge/data
      - ./reports:/traceforge/reports
      - ./samples:/traceforge/samples
    environment:
      - PYTHONUNBUFFERED=1
    stdin_open: true
    tty: true

  traceforge-dashboard:
    build: .
    container_name: traceforge-dashboard
    ports:
      - "5000:5000"
    volumes:
      - ./data:/traceforge/data
```

```
- ./reports:/traceforge/reports
environment:
  - PYTHONUNBUFFERED=1
entrypoint: ["/bin/bash", "-c"]
command: ["python3.11 -m dashboard.app"]
EOF
```

This command to be run on the terminal , not to be pasted on nano file.

Step 4.11 — Build the docker-compose.yml

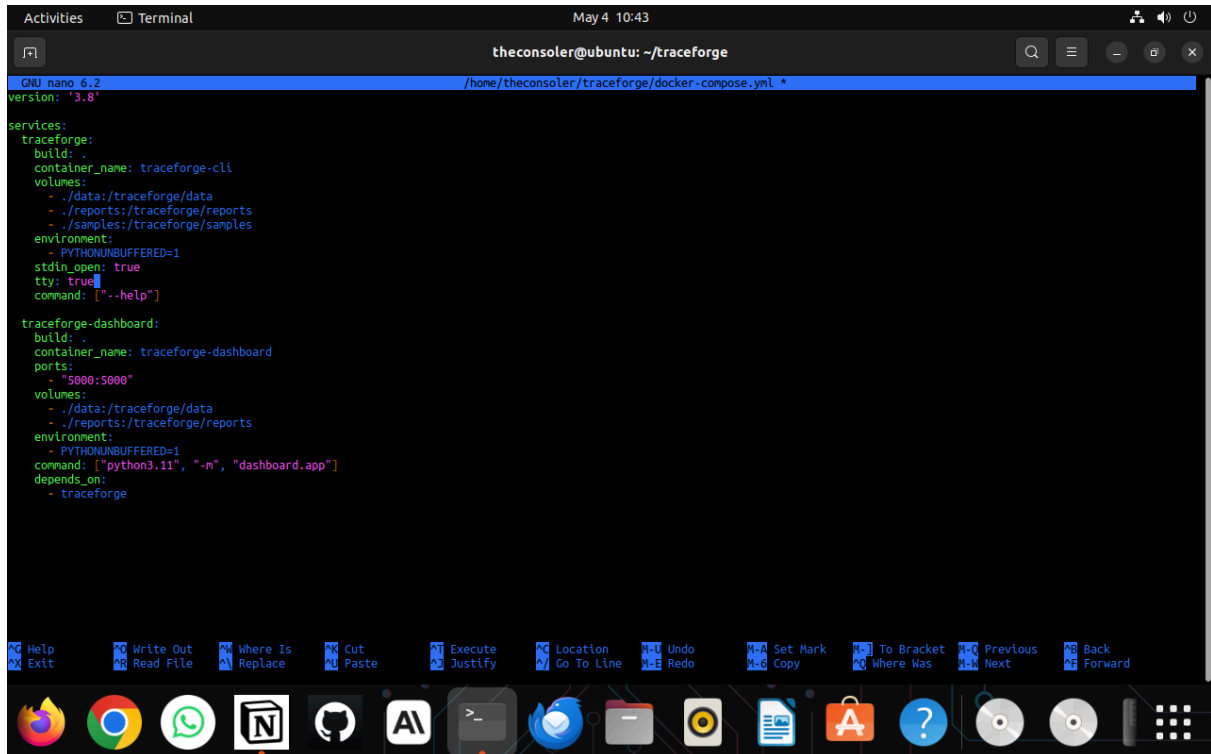
```
nano ~/traceforge/docker-compose.yml
```

```
version: '3.8'

services:
  traceforge:
    build: .
    container_name: traceforge-cli
    volumes:
      - ./data:/traceforge/data
      - ./reports:/traceforge/reports
      - ./samples:/traceforge/samples
    environment:
      - PYTHONUNBUFFERED=1
    stdin_open: true
    tty: true
    command: ["--help"]

  traceforge-dashboard:
    build: .
    container_name: traceforge-dashboard
    ports:
      - "5000:5000"
    volumes:
      - ./data:/traceforge/data
      - ./reports:/traceforge/reports
```

```
environment:
  - PYTHONUNBUFFERED=1
command: ["python3", "-m", "dashboard.app"]
depends_on:
  - traceforge
```



The screenshot shows a terminal window titled 'theconsoler@ubuntu: ~/traceforge' with a timestamp of 'May 4 10:43'. The terminal is running the GNU nano 6.2 editor, editing the file '/home/theconsoler/traceforge/docker-compose.yml'. The file content is as follows:

```
version: '3.8'

services:
  traceforge:
    build: .
    container_name: traceforge-cli
    volumes:
      - ./data:/traceforge/data
      - ./reports:/traceforge/reports
      - ./samples:/traceforge/samples
    environment:
      - PYTHONUNBUFFERED=1
    stdin_open: true
    tty: true
    command: ["--help"]

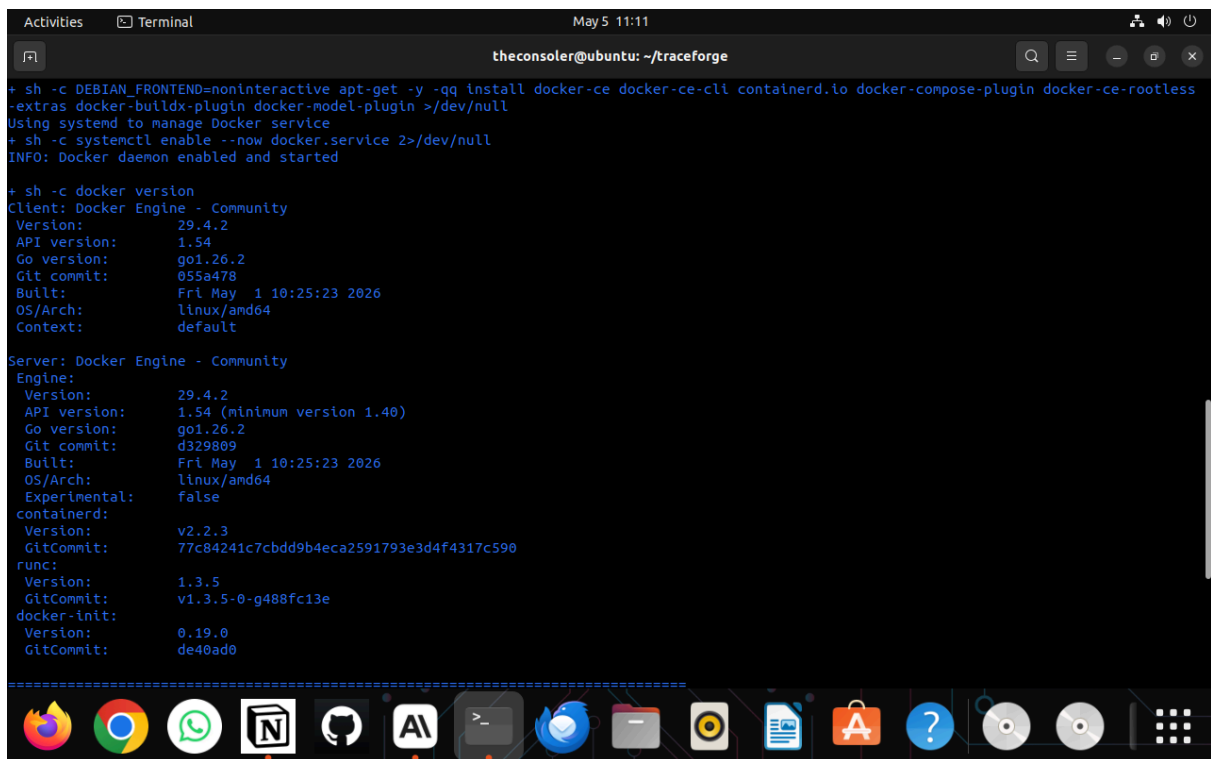
  traceforge-dashboard:
    build: .
    container_name: traceforge-dashboard
    ports:
      - "5000:5000"
    volumes:
      - ./data:/traceforge/data
      - ./reports:/traceforge/reports
    environment:
      - PYTHONUNBUFFERED=1
    command: ["python3.11", "-m", "dashboard.app"]
    depends_on:
      - traceforge
```

The terminal window includes a menu bar at the bottom with various editing and navigation options like 'Help', 'Exit', 'Write Out', 'Read File', 'Where Is', 'Replace', 'Cut', 'Paste', 'Execute', 'Justify', 'Location', 'Go To Line', 'Undo', 'Redo', 'Set Mark', 'Copy', 'To Bracket', 'Where Was', 'Previous', 'Next', 'Back', and 'Forward'.

Step 4.12 — Install, Build and test Docker

Step 1 — Install Docker:

```
curl -fsSL https://get.docker.com | sudo sh
```



The screenshot shows a terminal window titled 'theconsoler@ubuntu: ~/traceforge'. The user has executed a series of commands to install Docker. The first command is a one-liner that installs Docker CE, enables the service, and starts the daemon. Subsequent commands show the Docker client and server versions, along with details about the containerd and runc components. The terminal output is as follows:

```
+ sh -c DEBIAN_FRONTEND=noninteractive apt-get -y -qq install docker-ce docker-ce-cli containerd.io docker-compose-plugin docker-ce-rootless-extras docker-buildx-plugin docker-model-plugin >/dev/null
Using systemd to manage Docker service
+ sh -c systemctl enable --now docker.service 2>/dev/null
INFO: Docker daemon enabled and started

+ sh -c docker version
Client: Docker Engine - Community
 Version:           29.4.2
 API version:       1.54
 Go version:        go1.26.2
 Git commit:        055a478
 Built:             Fri May 1 10:25:23 2026
 OS/Arch:           linux/amd64
 Context:           default

Server: Docker Engine - Community
 Engine:
  Version:           29.4.2
  API version:       1.54 (minimum version 1.40)
  Go version:        go1.26.2
  Git commit:        d329809
  Built:             Fri May 1 10:25:23 2026
  OS/Arch:           linux/amd64
  Experimental:      false
 containerd:
  Version:           v2.2.3
  GitCommit:        77c84241c7cbdd9b4eca2591793e3d4f4317c590
 runc:
  Version:           1.3.5
  GitCommit:        v1.3.5-0-g488fc13e
 docker-init:
  Version:           0.19.0
  GitCommit:        de40ad0
```

Step 2 — Add your user to the docker group:

```
sudo usermod -aG docker $USER
```

Step 3 — Apply the group change:

```
newgrp docker
```

Step 4 — Verify Docker works:

```
docker --version
```

Step 5 — Now build TraceForge:

```
cd ~/traceforge
docker build -t traceforge:v2 .
```

```
Activities Terminal May 5 11:12
theconsoler@ubuntu: ~/traceforge

To run Docker as a non-privileged user, consider setting up the
Docker daemon in rootless mode for your user:

    dockerd-rootless-setuptool.sh install

Visit https://docs.docker.com/go/rootless/ to learn about rootless mode.

To run the Docker daemon as a fully privileged service, but granting non-root
users access, refer to https://docs.docker.com/go/daemon-access/

WARNING: Access to the remote API on a privileged Docker daemon is equivalent
to root access on the host. Refer to the 'Docker daemon attack surface'
documentation for details: https://docs.docker.com/go/attack-surface/

=====

theconsoler@ubuntu:~/traceforge$ sudo usermod -aG docker $USER
theconsoler@ubuntu:~/traceforge$ newgrp docker
theconsoler@ubuntu:~/traceforge$ docker --version
Docker version 29.4.2, build 055a478
theconsoler@ubuntu:~/traceforge$ docker build -t traceforge:v2 .
[+] Building 21.8s (3/11)
=> [internal] load build definition from Dockerfile
=> => transferring dockerfile: 1.03kB
=> [internal] load metadata for docker.io/library/ubuntu:22.04
=> [internal] load .dockerignore
=> => transferring context: 2B
=> [1/7] FROM docker.io/library/ubuntu:22.04@sha256:962f6cadeae0ea6284001009daa4cc9a8c37e75d1f5191cf0eb83fe565b63dd7
=> => resolve docker.io/library/ubuntu:22.04@sha256:962f6cadeae0ea6284001009daa4cc9a8c37e75d1f5191cf0eb83fe565b63dd7
=> => sha256:f63eb04151bcac21ad049f8d781b97b219aba392c5457907f8f3e88e43eb48ec 29.74MB / 29.74MB
=> => extracting sha256:f63eb04151bcac21ad049f8d781b97b219aba392c5457907f8f3e88e43eb48ec
=> [internal] load build context
=> => transferring context: 182.24MB
docker:default
0.0s
0.0s
7.7s
0.0s
0.0s
14.0s
0.0s
6.5s
7.4s
14.0s
14.0s
```

The build will take a few minutes the first time because it installs all system dependencies inside the container. Send me the output when it finishes.

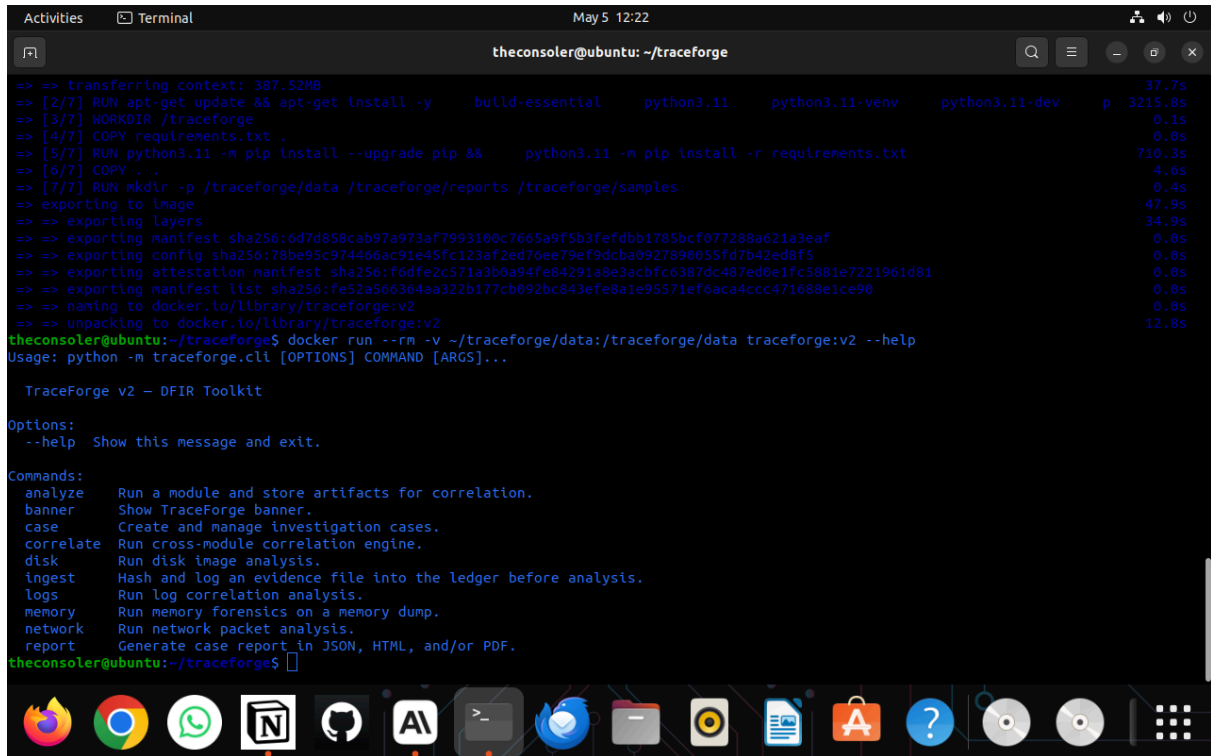
```
Activities Terminal May 5 12:18
theconsoler@ubuntu: ~/traceforge

=====

theconsoler@ubuntu:~/traceforge$ sudo usermod -aG docker $USER
theconsoler@ubuntu:~/traceforge$ newgrp docker
theconsoler@ubuntu:~/traceforge$ docker --version
Docker version 29.4.2, build 055a478
theconsoler@ubuntu:~/traceforge$ docker build -t traceforge:v2 .
[+] Building 4019.5s (12/12) FINISHED
=> [internal] load build definition from Dockerfile
=> => transferring dockerfile: 1.03kB
=> [internal] load metadata for docker.io/library/ubuntu:22.04
=> [internal] load .dockerignore
=> => transferring context: 2B
=> [1/7] FROM docker.io/library/ubuntu:22.04@sha256:962f6cadeae0ea6284001009daa4cc9a8c37e75d1f5191cf0eb83fe565b63dd7
=> => resolve docker.io/library/ubuntu:22.04@sha256:962f6cadeae0ea6284001009daa4cc9a8c37e75d1f5191cf0eb83fe565b63dd7
=> => sha256:f63eb04151bcac21ad049f8d781b97b219aba392c5457907f8f3e88e43eb48ec 29.74MB / 29.74MB
=> => extracting sha256:f63eb04151bcac21ad049f8d781b97b219aba392c5457907f8f3e88e43eb48ec
=> [internal] load build context
=> => transferring context: 387.52MB
=> [2/7] RUN apt-get update && apt-get install -y build-essential python3.11 python3.11-venv python3.11-dev p 3215.8s
=> [3/7] WORKDIR /traceforge
=> [4/7] COPY requirements.txt .
=> [5/7] RUN python3.11 -m pip install --upgrade pip && python3.11 -m pip install -r requirements.txt 710.3s
=> [6/7] COPY . .
=> [7/7] RUN mkdir -p /traceforge/data /traceforge/reports /traceforge/samples
=> exporting to image
=> => exporting layers
=> => exporting manifest sha256:6d7d858cab97a973af7993100c7665a9f5b3fefdbb1785bfc077288a621a3eaf
=> => exporting config sha256:78be95c974466ac91e45fc123af2ed76ee79ef9dcba0927890055fd7b42ed8f5
=> => exporting attestation manifest sha256:f6dfe2c571a3b0a94fe84291a8e3acbf6387dc487ed0e1fc5881e7221961d81
=> => exporting manifest list sha256:fe52a566364aa322b177cb092bc843ef8a1e95571ef6aca4ccc471688e1ce90
=> => naming to docker.io/library/traceforge:v2
=> => unpacking to docker.io/library/traceforge:v2
theconsoler@ubuntu:~/traceforge$
```

Test the CLI works in Docker:


```
docker run --rm -v ~/traceforge/data:/traceforge/data traceforge:v2 --help
```



```
theconsoler@ubuntu: ~/traceforge
=> == transferring context: 387.52MB
=> [2/7] RUN apt-get update && apt-get install -y build-essential python3.11 python3.11-venv python3.11-dev p 3215.8s
=> [3/7] WORKDIR /traceforge
=> [4/7] COPY requirements.txt .
=> [5/7] RUN python3.11 -m pip install --upgrade pip && python3.11 -m pip install -r requirements.txt 710.3s
=> [6/7] COPY . .
=> [7/7] RUN mkdir -p /traceforge/data /traceforge/reports /traceforge/samples
=> exporting to image
=> == exporting layers
=> == exporting manifest sha256:6d7d858cab97a973af7993100c7665a9f5b3fefdbb1785bcf077288a621a3eaf 0.0s
=> == exporting config sha256:78be95c974466ac91e45fc123af2ed70ee79ef9dcba0927890055fd7b42ed8f5 0.0s
=> == exporting attestation manifest sha256:f6dfe2c571a3b0a94fe84291a8e3acbf6387dc487ed0e1fc5881e7221961d81 0.0s
=> == exporting manifest list sha256:fe52a566364aa322b177cb092bc843efe8a1e95571ef6aca4ccc471688e1ce90 0.0s
=> == naming to docker.io/library/traceforge:v2
=> == unpacking to docker.io/library/traceforge:v2 12.8s
theconsoler@ubuntu:~/traceforge$ docker run --rm -v ~/traceforge/data:/traceforge/data traceforge:v2 --help
Usage: python -m traceforge.cli [OPTIONS] COMMAND [ARGS]...

TraceForge v2 - DFIR Toolkit

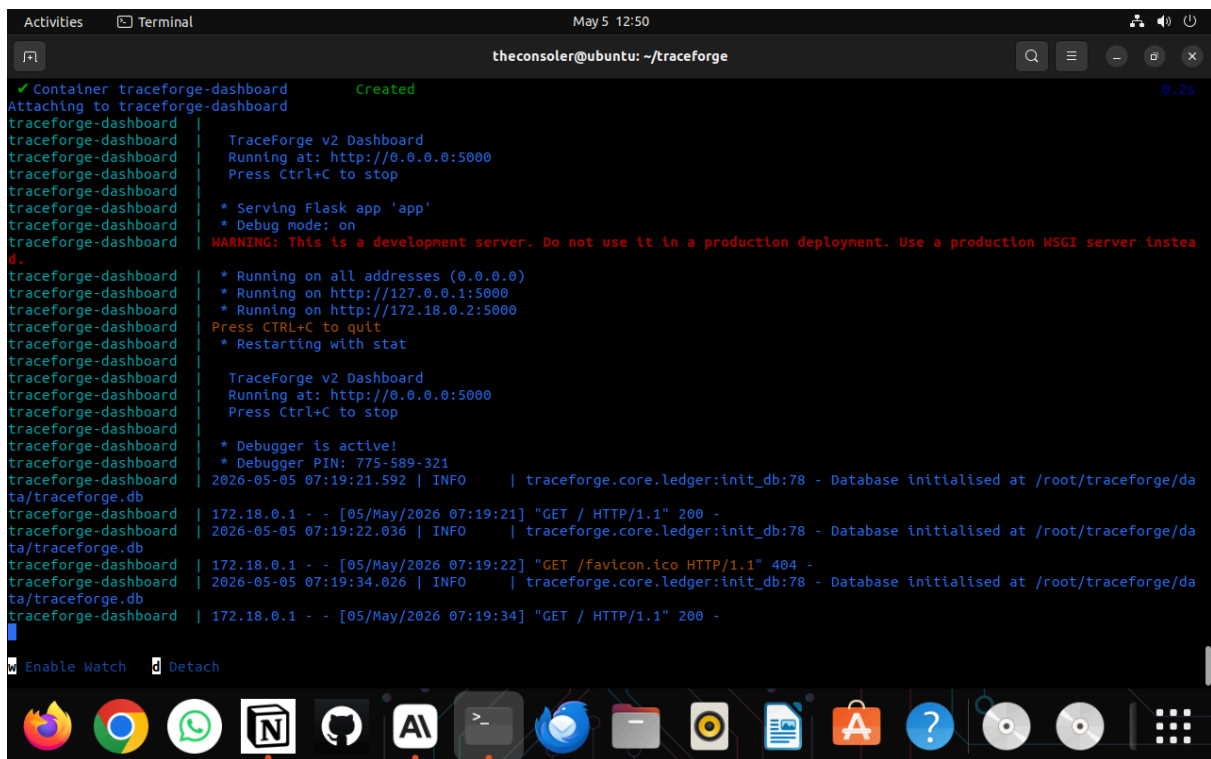
Options:
  --help  Show this message and exit.

Commands:
  analyze  Run a module and store artifacts for correlation.
  banner   Show TraceForge banner.
  case     Create and manage investigation cases.
  correlate Run cross-module correlation engine.
  disk     Run disk image analysis.
  ingest   Hash and log an evidence file into the ledger before analysis.
  logs     Run log correlation analysis.
  memory   Run memory forensics on a memory dump.
  network  Run network packet analysis.
  report    Generate case report in JSON, HTML, and/or PDF.
theconsoler@ubuntu:~/traceforge$
```

Test the dashboard works in Docker:

```
docker-compose up traceforge-dashboard
```

Open browser at <http://127.0.0.1:5000>



```
theconsoler@ubuntu: ~/traceforge
Container traceforge-dashboard Created
Attaching to traceforge-dashboard
traceforge-dashboard | TraceForge v2 Dashboard
traceforge-dashboard | Running at: http://0.0.0.0:5000
traceforge-dashboard | Press Ctrl+C to stop
traceforge-dashboard | * Serving Flask app 'app'
traceforge-dashboard | * Debug mode: on
traceforge-dashboard | WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
traceforge-dashboard | * Running on all addresses (0.0.0.0)
traceforge-dashboard | * Running on http://127.0.0.1:5000
traceforge-dashboard | * Running on http://172.18.0.2:5000
traceforge-dashboard | Press CTRL+C to quit
traceforge-dashboard | * Restarting with stat
traceforge-dashboard | TraceForge v2 Dashboard
traceforge-dashboard | Running at: http://0.0.0.0:5000
traceforge-dashboard | Press Ctrl+C to stop
traceforge-dashboard | * Debugger is active!
traceforge-dashboard | * Debugger PIN: 775-589-321
traceforge-dashboard | 2026-05-05 07:19:21.592 | INFO | traceforge.core.ledger:init_db:78 - Database initialised at /root/traceforge/data/traceforge.db
traceforge-dashboard | 172.18.0.1 - - [05/May/2026 07:19:21] "GET / HTTP/1.1" 200 -
traceforge-dashboard | 2026-05-05 07:19:22.036 | INFO | traceforge.core.ledger:init_db:78 - Database initialised at /root/traceforge/data/traceforge.db
traceforge-dashboard | 172.18.0.1 - - [05/May/2026 07:19:22] "GET /favicon.ico HTTP/1.1" 404 -
traceforge-dashboard | 2026-05-05 07:19:34.026 | INFO | traceforge.core.ledger:init_db:78 - Database initialised at /root/traceforge/data/traceforge.db
traceforge-dashboard | 172.18.0.1 - - [05/May/2026 07:19:34] "GET / HTTP/1.1" 200 -
```

To stop docker-compose up use and then use build command to make it again then start it again.

```
docker-compose down
docker rmi traceforge-traceforge-dashboard traceforge-traceforge traceforge:v2
docker build -t traceforge:v2 .
docker-compose up traceforge-dashboard
```

Docker is working perfectly now. Looking at the output:

```
* Running on all addresses (0.0.0.0)
* Running on http://127.0.0.1:5000
* Running on http://172.18.0.2:5000
GET / HTTP/1.1" 200 ← requests are being served successfully
```

Here's what each IP means:

0.0.0.0 — means Flask is listening on all network interfaces. This is the binding address, not something you visit in a browser.

127.0.0.1:5000 — localhost. Use this from the browser on the same machine. This is what you open in Chrome.

`172.18.0.2:5000` — this is the Docker container's internal IP address on the Docker virtual network. Other containers in the same docker-compose network can reach the dashboard using this IP. You don't need this for normal use.

The `GET / HTTP/1.1" 200` at the bottom confirms the dashboard is serving pages successfully — 200 means OK.

So for your daily use:

- Open browser at `http://127.0.0.1:5000` — always works
- Share `http://192.168.223.140:5000` with teammates on your local network

Step 4.13 — Final commit and push

```
cd ~/traceforge
git add .
git commit -m "feat: phase 4 complete – Flask dashboard, Dockerfile, docker-compose"
git push origin main
```

Phase 4 completion checklist

- ☒ Flask app created — `dashboard/app.py`
- ☒ All four templates created — `base, index, case, timeline, correlations`
- ☒ Dashboard runs at `http://127.0.0.1:5000`
- ☒ Cases display on home page
- ☒ Timeline page shows filterable events
- ☒ Correlations page shows findings
- ☒ Report export buttons work
- ☒ Dockerfile builds without errors
- ☒ `docker-compose.yml` created
- ☒ Final commit pushed to GitHub

When every box is checked: TraceForge v2 is complete.

TraceForge v2 build documentation — Phase 4 of 4 Last updated: May 2026